CSS BÀSIC



Jaume Joan Pericàs Oliver
poj@paucasesnovescifp.cat

CONTINGUTS

1. Introducció al CSS
2. Aplicació d'estils
3. Sintaxi bàsica del CSS
4. Propietats de text i fonts
5. Marges, farcits i vores
6. Colors i fons
7. Dimensions i caixes
8. Posicionament bàsic
9. Disseny bàsic amb Flexbox
10. Media Queries (Introducció)

Introducció al CSS

Què és el CSS?

CSS (Cascading Style Sheets) és un llenguatge de fulles d'estil que s'utilitza per controlar la presentació i el disseny dels documents web escrits en HTML. Mentre que HTML proporciona l'estructura del contingut, CSS defineix com s'ha de visualitzar aquest contingut en termes de disseny, colors, tipografia, espaiat i posicionament dels elements en una pàgina.



Introducció al CSS

Importància del CSS en el desenvolupament web

El CSS és una peça fonamental del desenvolupament web, ja que permet separar la presentació visual del contingut estructural d'una pàgina. Això ofereix una sèrie d'avantatges:

1. **Disseny i estètica millorats**: Amb CSS, pots aplicar estils atractius i professionals a les pàgines web, fent-les visualment atractives i més fàcils de llegir i navegar.
2. **Separació de contingut i estil**: Gràcies al CSS, és possible mantenir els estils de presentació (com colors, mides de text, marges) fora del codi HTML, el que facilita el manteniment i l'actualització del disseny sense afectar l'estructura del contingut.
3. **Facilitat de manteniment**: Amb un sol fitxer CSS, pots aplicar estils a múltiples pàgines web, estalviant temps i esforç quan es fan canvis de disseny. Això permet una actualització i un manteniment molt més eficient.
4. **Millor experiència d'usuari**: Amb el CSS es poden crear dissenys adaptables (responsive) que funcionen bé en diferents dispositius, com mòbils, tauletes i ordinadors de sobretaula. Aquest tipus de disseny millora significativament l'experiència de l'usuari.
5. **Rendiment i velocitat**: Utilitzar fitxers CSS externs pot ajudar a carregar les pàgines web més ràpidament, ja que els navegadors poden **emmagatzemar en memòria cau aquests fitxers i reutilitzar-los en diverses pàgines**, estalviant ample de banda i temps de càrrega.

Aplicació d'estils

El CSS es pot aplicar a un document HTML de tres maneres principals: **en línia**, **incrustat** (dins de l'HTML) i **extern** (mitjançant un fitxer CSS separat). Cadascuna d'aquestes formes té els seus propòsits i usos.

1. Estils en línia (Inline styles)
Els estils en línia s'apliquen directament a un element HTML, utilitzant l'atribut style dins de la mateixa etiqueta de l'element. Aquest mètode és adequat per a aplicar estils a un sol element de manera ràpida, però no és recomanable per a pàgines grans, ja que dificulta la mantenibilitat.

```
<p style="color: red; font-size: 20px;">Aquest és un paràgraf amb estils en línia.</p>
```

Avantatges:

1. Aplicació ràpida i directa a l'element.



Inconvenients:

1. Dificulta la mantenibilitat en llocs webs grans perquè caldria aplicar l'estil a cada element manualment.
2. No aprofita la capacitat de reutilitzar els estils.

Aplicació d'estils

2. Estils incrustats (Embedded styles)

Els estils incrustats es col·loquen dins de la secció <head> d'un document HTML, dins d'una etiqueta <style>. Aquests estils s'apliquen a tots els elements HTML de la pàgina actual.

```html
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <title>Estils incrustats</title>
  <style>
    p {
      color: blue;
      font-size: 18px;
    }
    h1 {
      text-align: center;
    }
  </style>
</head>
<body>
  <h1>Aquest és un títol centrat</h1>
  <p>Aquest paràgraf tindrà text de color blau i mida 18 píxels.</p>
</body>
</html>
```



Avantatges:

1. Més eficient que els estils en línia, ja que es poden aplicar estils a diversos elements d'una mateixa pàgina des d'un sol lloc.



Inconvenients:

1. Només afecta la pàgina on s'aplica, no es reutilitza entre diverses pàgines.
2. Si la pàgina creix massa, pot fer més difícil la lectura i manteniment.

Aplicació d'estils

3. Estils externs (External stylesheets)

Els estils externs s'emmagatzemen en un fitxer CSS separat, i s'enllacen al document HTML utilitzant l'etiqueta <link> dins de la secció <head>. Aquest és el mètode més recomanat per a la majoria de llocs webs, especialment si consten de múltiples pàgines.

```html
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <title>Estils externs</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Benvinguts al meu lloc web</h1>
  <p>Aquest paràgraf tindrà text verd i mida 16 píxels.</p>
</body>
</html>
```

```css
p {
  color: green;
  font-size: 16px;
}

h1 {
  text-align: center;
  color: darkblue;
}
```

Inconvenients:

1. Si el fitxer CSS extern no es carrega correctament (per problemes de connexió o d'enllaç), la pàgina es mostrarà sense estils.

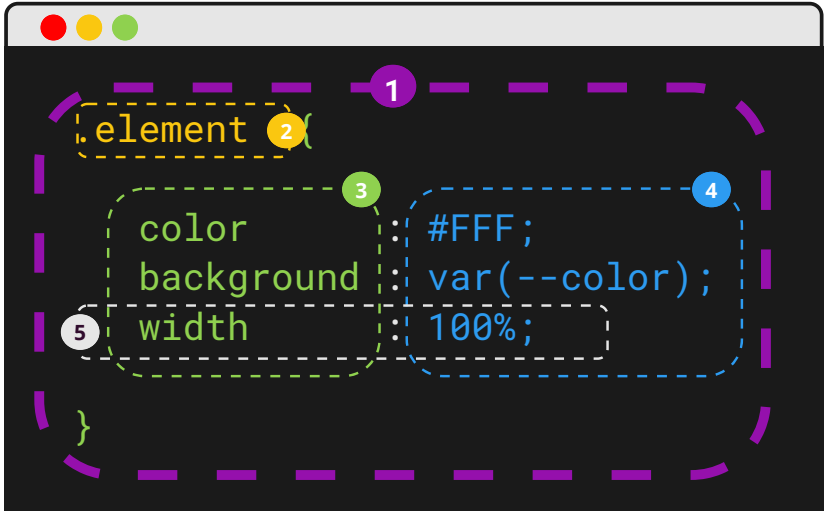




Avantatges:

1. Permet reutilitzar els estils en diverses pàgines web, facilitant la consistència i mantenibilitat.
2. Facilita la separació entre el contingut (HTML) i l'estil (CSS).
3. Optimitza el temps de càrrega del lloc web, ja que els navegadors poden emmagatzemar en memòria cau els fitxers CSS.

Sintaxi bàsica del CSS



1

2 **Selector**: Indica l'element HTML al qual s'aplicaran els estils.

3 **Propietat**: Defineix la característica que vols modificar (com ara el color, la mida del text, el marge, etc.).

4 **Valor**: Assigna un valor específic a la propietat (com ara "blau" per al color, "16px" per a la mida del text, etc.).

5 **Declaració CSS** és una instrucció dins d'una regla CSS que defineix una característica específica d'estil per a un element HTML.

Selectors en CSS

Els **selectors** són una part fonamental del CSS perquè defineixen quins elements HTML seran afectats per les regles d'estil. Hi ha diversos tipus de selectors, però els més comuns en l'aprenentatge bàsic són:

1. **Selector de tipus (Type Selector)**
Aquest selector aplica estils a tots els elements HTML del mateix tipus. Aquest codi aplicarà el color verd a tots els títols (<h1>) de la pàgina.

```
h1 {
 color: green;
}
```

2. **Selector de classe (Class Selector)**
Els selectors de classe permeten aplicar estils a elements HTML amb una classe específica. S'utilitza un punt (.) per seleccionar una classe. Qualsevol element HTML que tingui l'atribut class="mi-classe" tindrà un fons groc. Per exemple:

```
.mi-classe {
  background-color: yellow;
}
```

```
<p class="mi-classe">Aquest paràgraf tindrà un fons groc.</p>
```

Selectors en CSS

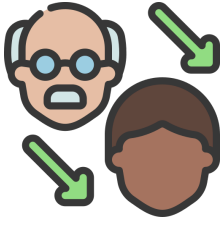
3. **Selector d'id (ID Selector)**
El selector d'id s'utilitza per aplicar estils a un únic element HTML que tingui un identificador específic. Es fa servir el símbol de la coixinera #. Aquest exemple centrarà el text de l'element que tingui id="mi-id". És important recordar que un ID ha de ser únic a la pàgina.

```
#mi-id {
 text-align: center;
}
```

4. **Selector universal**
El selector universal (*) aplica estils a **tots els elements** d'una pàgina. Aquest exemple elimina els marges i farcits de tots els elements de la pàgina, cosa útil per establir un estil uniforme.

```
* {
  margin: 0;
  padding: 0;
}
```

Conceptes importants



Herència

Els fills hereten els estils dels seus pares, així no cal que escriguis estils per a tot.



Especificitat

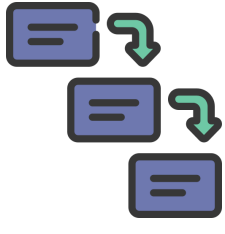
Quan hi ha conflictes d'estils, el navegador aplica només el de més especificitat.

```
.parrafo {
    color: blue;
}

.article .parrafo{
    color:red;
}
```

```
body {
    color: blue;
}

h1{
    color:blue;
}
```



Cascada

Tot estil sobreescriu un d'anterior. La C de CSS significa Cascada.

```
.parrafo {
    color: blue;
}

.parrafo{
    color:red;
}
```

Propietats de text i fonts

El CSS permet controlar com es mostren els textos dins d'una pàgina web. Amb diverses propietats, podem canviar la font, el color, l'alineació, la mida del text, i altres aspectes visuals per tal de millorar la llegibilitat i l'estètica del contingut.

1. Propietat font-family

La propietat font-family defineix el tipus de lletra (tipografia) que s'utilitzarà per a un element de text. Pots especificar una o més fonts, i el navegador aplicarà la primera que estigui disponible al sistema de l'usuari.

```
p {
  font-family: "Arial", sans-serif;
}
```

2. Propietat font-size

La propietat font-size controla la mida de la lletra. Els valors es poden definir en diverses unitats, com píxels (px), ems (em), punts (pt), o percentatges (%).

```
h1 {
  font-size: 32px;
}
p {
  font-size: 14px;
}
```

Propietats de text i fonts

3. Propietat color

La propietat color defineix el color del text. Es poden utilitzar noms de colors predefinits, valors en hexadecimal, o el sistema de colors RGB o RGBA.

```
p {
  color: red; /* Amb nom de color */
}

h1 {
  color: #00FF00; /* Amb codi hexadecimal */
}

span {
  color: rgb(0, 0, 255); /* Amb codi RGB */
}
```

4. Propietat text-align

La propietat text-align s'utilitza per controlar l'alineació del text dins d'un element. Els valors més comuns són left (esquerra), right (dreta), center (centrat) i justify (justificat).

```
h1 {
  text-align: center;
}
p {
  text-align: justify;
}
```

Propietats de text i fonts

5. Propietat font-weight

La propietat font-weight controla el gruix de la lletra. Els valors comuns són:

- normal: per defecte.
- bold: text en negreta.
- Números entre 100 i 900, on 400 és el valor per defecte i 700 representa negreta.

```
p {
  font-weight: bold;
}
```

6. Propietat font-style

La propietat font-style controla l'estil de la font, com si el text serà normal o en cursiva.
Els valors més comuns són:

- normal: Text normal.
- italic: Text en cursiva.
- oblique: Text inclinat.

```
span {
  font-style: italic;
}
```

Màrges, farcits i vores en CSS

Els **marges** (margin), els **farcits** (padding) i les **vores** (border) són part del **model de caixa** en CSS, que descriu com es disposen els elements en la pàgina. Aquestes propietats permeten ajustar l'espai exterior i interior d'un element, així com definir-ne els límits visuals amb una vora.

1. Marges (margin)
La propietat margin controla l'espai exterior d'un element, és a dir, l'espai que hi ha entre un element i els altres elements que l'envolten. Els marges es poden definir de manera individual per a cadascun dels quatre costats de l'element (superior, dret, inferior, esquerre) o de manera global per a tots els costats alhora.

```
p {
  margin: 10px; /* Aplica un marge de 10 píxels a tots els costats */
}

h1 {
  margin-top: 20px; /* Marge superior de 20 píxels */
  margin-bottom: 10px; /* Marge inferior de 10 píxels */
  margin-left: 10px; /* Marge esquerra de 10 píxels */
  margin-right: 10px; /* Marge dret de 10 píxels */
}
```

Màrges, farcits i vores en CSS

```
div {
  padding: 20px; /* Aplica un farcit de 20 píxels a tots els costats */
}

h2 {
  padding-left: 15px; /* Aplica un farcit de 15 píxels al costat esquerre */
}
```

```
div {
  border: 2px solid black; /* Vora de 2 píxels, sòlida i negra */
}

p {
  border: 1px dashed blue; /* Vora d'1 píxel, ratllada i blava */
}
```

3. Vores (border)

La propietat border s'utilitza per afegir una vora al voltant d'un element. Aquesta propietat té diversos subatributs:

1. **amplada de la vora** (border-width): defineix el gruix de la vora.
2. **estil de la vora** (border-style): pot ser solid (sòlid), dashed (ratllat), dotted (puntets), etc.
3. **color de la vora** (border-color): especifica el color de la vora.

Màrges, farcits i vores en CSS

```html
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Màrges, Farcits i Vores</title>
  <style>
    .caixa {
      margin: 20px; /* Marge de 20 píxels a tots els costats */
      padding: 15px; /* Farcit de 15 píxels a tots els costats */
      border: 3px solid red; /* Vora de 3 píxels, sòlida i vermella */
    }

    .titol {
      margin-top: 30px; /* Marge superior de 30 píxels */
      padding-bottom: 10px; /* Farcit inferior de 10 píxels */
      border-bottom: 2px dashed green; /* Vora inferior ratllada i verda */
    }
  </style>
</head>
<body>
  <div class="caixa">
    <h2 class="titol">Títol amb marges i vores</h2>
    <p>Aquest és un paràgraf dins d'una caixa amb marges, farcits i una vora sòlida.</p>
  </div>
</body>
</html>
```

Unitats comunes:

1. **Píxels (px)**:
   a. Unitat fixa, representa un punt a la pantalla.
   b. **Exemple**: margin: 20px;
2. **Ems (em)**:
   a. Unitat **relativa** a la mida de la font de l'element pare.
   b. **Exemple**: padding: 2em; (2 vegades la mida de la font).
3. **Rems (rem)**:
   a. Unitat **relativa** a la mida de la font de l'element arrel (html).
   b. **Exemple**: margin: 1.5rem; (1.5 vegades la mida de la font arrel).
4. **Percentatge (%)**:
   a. Relativa a l'amplada o alçada del contenidor pare.
   b. **Exemple**: padding: 5%; (5% de l'amplada del contenidor).
5. **Viewports (vw, vh)**:
   a. Relativa a la mida de la finestra visible.
   b. **Exemple**: margin: 5vw; (5% de l'amplada del viewport).
6. **Vores (border-width)**:
   a. Gruixos predeterminats: thin, medium, thick o valors com px.
   b. **Exemple**: border: thick solid black;

Colors i fons en CSS

El CSS proporciona diverses propietats per aplicar colors i fons als elements HTML. Aquestes propietats són essencials per millorar l'aspecte visual i la llegibilitat de la pàgina web.

1. Propietat color

La propietat color s'utilitza per canviar el color del text dins d'un element. Pots definir els colors de diverses maneres:

1. **Noms de colors**: utilitzar noms predefinits com red, blue, green, etc.
2. **Colors hexadecimals**: utilitzar un codi hexadecimal que comença amb # seguit de sis caràcters (exemple: #FF0000 per a vermell).
3. **RGB**: utilitzar valors de rgb() on cada component (vermell, verd, blau) s'especifica entre 0 i 255.
4. **RGBA**: igual que rgb(), però amb un quart valor per a l'opacitat (valors entre 0.0 i 1.0).

```
p {
  color: blue; /* Utilitza un nom de color */
}

h1 {
  color: #FF5733; /* Utilitza un codi hexadecimal */
}

span {
  color: rgb(0, 255, 0); /* Utilitza el sistema RGB */
}

div {
  color: rgba(0, 0, 255, 0.5); /* Utilitza el sistema RGBA amb
      opacitat */
}
```

Colors i fons en CSS

2. Propietat background-color

La propietat background-color s'utilitza per establir el color de fons d'un element. Igual que la propietat color, pots definir colors utilitzant noms, valors hexadecimals, RGB o RGBA.

```
body {
  background-color: lightgray; /* Color de fons gris clar */
}

div {
  background-color: #00FF00; /* Color de fons verd */
}
```

3. Propietat background

La propietat background és una manera abreujada d'especificar diverses propietats de fons alhora, com el color, la imatge de fons, la repetició, la posició, etc.

```
div {
  background: lightblue url("fons.jpg") no-repeat center cover;
}
```

Colors i fons en CSS

4. Propietat background-image
Aquesta propietat permet afegir una imatge com a fons d'un element. Pots especificar la ubicació de la imatge amb una URL.

```
div {
  background-image: url("imatge-de-fons.jpg");
}
```

Si vols que la imatge es repeteixi o es mostri d'una altra manera, pots utilitzar altres propietats complementàries:

1. **background-repeat**: Controla si la imatge de fons es repeteix. Els valors poden ser repeat, no-repeat, repeat-x (horitzontalment) o repeat-y (verticalment).
2. **background-position**: Defineix la posició de la imatge de fons (per exemple, center, top, left).
3. **background-size**: Permet ajustar la mida de la imatge (per exemple, cover o contain per adaptar-se a la mida de l'element).

```
div {
  background-image: url("fons.jpg");
  background-repeat: no-repeat;
  background-position: center;
  background-size: cover;
}
```

Colors i fons en CSS

Resum de les propietats vistes:

1. **color**: Defineix el color del text.
2. **background-color**: Estableix el color de fons d'un element.
3. **background-image**: Afegeix una imatge de fons a un element.
4. **background-repeat**: Controla la repetició de la imatge de fons.
5. **background-position**: Defineix la posició de la imatge de fons.
6. **background-size**: Ajusta la mida de la imatge de fons.
7. **background**: Propietat abreujada que combina diverses propietats de fons.

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Colors i Fons en CSS</title>
  <style>
    body {
      background-color: lightgray; /* Color de fons del cos de la pàgina */
    }

    h1 {
      color: darkblue; /* Color del text del títol */
    }

    p {
      color: #333; /* Color de text amb codi hexadecimal */
      background-color: #FFD700; /* Fons groc daurat */
    }

    .fons-imatge {
      background-image: url("fons.jpg");
      background-repeat: no-repeat;
      background-position: center;
      background-size: cover;
    }
  </style>
</head>
<body>
  <h1>Colors i Fons en CSS</h1>
  <p>Aquest és un paràgraf amb text de color gris fosc i fons groc daurat.</p>
  <div class="fons-imatge" style="height: 200px;">
    Aquest div té una imatge de fons.
  </div>
</body>
</html>
```

Dimensions i caixes en CSS

El CSS utilitza el **model de caixa** per representar els elements d'una pàgina web. Cada element es considera com una caixa rectangular que conté el contingut, farcits, marges i vores. És important entendre aquest model perquè controla com es distribueixen i s'ajusten els elements dins de la pàgina.

El model de caixa es compon de quatre parts principals:

1. **Contingut**: El contingut real de l'element, com text, imatges o altres elements.
2. **Farcit (padding)**: L'espai entre el contingut i la vora.
3. **Vora (border)**: El límit al voltant del farcit.
4. **Marge (margin)**: L'espai extern entre la vora i altres elements.

Dimensions i caixes en CSS

1. Propietats de dimensions (width i height)

Les propietats width (amplada) i height (alçada) s'utilitzen per definir la mida d'un element. Aquests valors es poden definir en unitats fixes com píxels (px) o unitats relatives com percentatges (%), que depenen de la mida del seu element contenidor.

```
div {
  width: 300px;  /* Amplada fixa de 300 píxels */
  height: 150px; /* Alçada fixa de 150 píxels */
}

.container {
  width: 50%; /* Amplada del 50% de l'element contenidor */
  height: auto; /* L'alçada s'ajusta automàticament */
}
```

Dimensions i caixes en CSS

2. El model de caixa en CSS

El **model de caixa** estableix com es calculen les dimensions d'un element tenint en compte les propietats de marges, farcits, i vores. Per defecte, la mida de width i height només afecta el contingut. Per incloure el farcit i la vora dins de les dimensions de l'element, es pot utilitzar la propietat box-sizing.

Propietat box-sizing

La propietat box-sizing controla com es calculen les dimensions totals de l'element. Hi ha dues opcions principals:

1. **content-box** (valor per defecte): Només inclou el contingut en les dimensions, mentre que les vores i els farcits s'afegeixen a part.
2. **border-box**: Inclou el contingut, el farcit i la vora dins de les dimensions especificades.

```
div {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: border-box;
}
```

Dimensions i caixes en CSS

3. Control del desbordament (overflow)

Quan el contingut d'un element és més gran que les dimensions especificades, el CSS ofereix la propietat overflow per controlar com es mostra el contingut que desborda.

Els valors més comuns són:

- **visible**: El contingut que desborda es mostra fora de l'element (valor per defecte).
- **hidden**: El contingut que desborda es retalla i no es mostra.
- **scroll**: Mostra barres de desplaçament, encara que el contingut no desbordi.
- **auto**: Mostra barres de desplaçament només si el contingut desborda.

```
div {
  width: 150px;
  height: 100px;
  overflow: scroll;
}
```

Dimensions i caixes en CSS

Resum de les propietats vistes:

1. **width i height**: Defineixen les dimensions d'un element.
2. **box-sizing**: Controla com es calculen les dimensions totals d'un element tenint en compte el farcit i la vora.
3. **overflow**: Controla com es maneja el contingut que desborda les dimensions de l'element.

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dimensions i Caixes en CSS</title>
  <style>
    .caixa {
      width: 300px;
      height: 200px;
      padding: 20px;
      border: 5px solid darkblue;
      margin: 20px;
      box-sizing: border-box;
    }

    .desbordament {
      width: 150px;
      height: 100px;
      border: 2px solid red;
      overflow: scroll;
    }
  </style>
</head>
<body>
  <div class="caixa">
    Aquesta és una caixa amb farcit i vora. Les dimensions totals es calculen amb
        `box-sizing: border-box`.
  </div>

  <div class="desbordament">
    Aquest és un exemple amb desbordament. Si el contingut és més gran que l'espai
        disponible, es mostraran barres de desplaçament.
  </div>
</body>
</html>
```

Posicionament bàsic en CSS

El CSS ofereix diverses tècniques per controlar la ubicació dels elements en una pàgina web. La propietat clau per al posicionament és position, que té diversos valors per a definir com es posiciona un element respecte al seu element contenidor o al document.

1. La propietat position

La propietat position permet especificar el mètode de posicionament d'un element. Els valors més comuns són:

1. **static** (per defecte): Els elements es posicionen de manera normal, seguint el flux del document. No es pot ajustar manualment amb coordenades.
2. **relative**: Es posiciona l'element respecte a la seva posició original en el flux del document. Es poden utilitzar coordenades per moure'l lleugerament (dalt, baix, esquerra o dreta).
3. **absolute**: L'element es treu del flux del document i es posiciona respecte al seu element contenidor més proper que tingui una posició definida. Si no hi ha cap contenidor definit, es posiciona respecte a la pàgina.
4. **fixed**: L'element es treu del flux del document i es posiciona respecte a la finestra del navegador. Es manté fix fins i tot quan l'usuari fa desplaçaments.
5. **sticky**: L'element es comporta com relative fins que arriba a un punt en el desplaçament de la pàgina, després del qual es comporta com fixed.

Posicionament bàsic en CSS

2. Propietats de posicionament (top, right, bottom, left)

Quan es defineix la propietat position com relative, absolute, fixed o sticky, pots utilitzar les propietats top, right, bottom, i left per especificar la distància de l'element respecte als seus límits.

Exemples:

- **top: 10px;**: Moure l'element 10 píxels cap avall des del límit superior.
- **right: 20px;**: Moure l'element 20 píxels cap a l'esquerra des del límit dret.

```
div {
  position: relative;
  top: 20px; /* Mou l'element 20 píxels cap avall des de la seva posició original */
  left: 30px; /* Mou l'element 30 píxels cap a la dreta */
}
```

Posicionament bàsic en CSS

3. Posicionament absolut (position: absolute)

Amb position: absolute, l'element es treu completament del flux normal del document i es posiciona respecte al seu element contenidor més proper amb position: relative, absolute, o fixed.

```
div {
  position: absolute;
  top: 50px;
  left: 100px;
}
```

4. Posicionament fix (position: fixed)

Els elements amb position: fixed es mantenen en una ubicació fixa a la pantalla fins i tot quan l'usuari fa desplaçaments. Es posicionen respecte a la finestra del navegador.

```
div {
  position: fixed;
  top: 0;
  left: 0;
  width: 100px;
  height: 100px;
  background-color: red;
}
```

Posicionament bàsic en CSS

5. Posicionament enganxós (position: sticky)

Els elements amb position: sticky es comporten com relative fins que l'usuari es desplaça per la pàgina i arriba a un punt especificat, després del qual l'element es comporta com fixed.

```
h1 {
  position: sticky;
  top: 10px;
}
```

6. Propietat z-index

La propietat z-index es fa servir per controlar l'ordre de superposició dels elements posicionats (relative, absolute, fixed o sticky). Els valors més alts de z-index col·loquen l'element davant d'altres elements amb valors inferiors.

```
div {
  position: absolute;
  z-index: 10; /* Aquest element es
      mostrarà davant dels altres amb
      z-index inferior */
}
```

Posicionament bàsic en CSS

```html
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Posicionament en CSS</title>
  <style>
    .relatiu {
      position: relative;
      top: 20px;
      left: 30px;
      background-color: lightblue;
      padding: 10px;
    }

    .absolut {
      position: absolute;
      top: 50px;
      left: 50px;
      background-color: lightgreen;
      padding: 10px;
    }

    .fix {
      position: fixed;
      top: 0;
      right: 0;
      background-color: coral;
      padding: 10px;
    }

    .enganxós {
      position: sticky;
      top: 10px;
      background-color: yellow;
      padding: 10px;
    }
  </style>
</head>
<body>
  <h1 class="enganxós">Títol enganxós</h1>
  <div class="relatiu">Aquest element té `position: relative`.</div>
  <div class="absolut">Aquest element té `position: absolute`.</div>
  <div class="fix">Aquest element té `position: fixed`.</div>
</body>
</html>
```

Resum de les propietats vistes:

1. **position: static**: Posicionament per defecte.
2. **position: relative**: L'element es mou respecte a la seva posició original.
3. **position: absolute**: L'element es posiciona respecte al seu contenidor més proper.
4. **position: fixed**: L'element es fixa respecte a la finestra del navegador.
5. **position: sticky**: L'element es comporta com relative fins que arriba a una posició fixada, on es manté enganxós.
6. **z-index**: Controla l'ordre de superposició dels elements.

Disseny bàsic amb Flexbox

Flexbox és un sistema de disseny en CSS que proporciona una manera molt eficient de distribuir l'espai entre els elements i alinear-los dins d'un contenidor, fins i tot quan la mida dels elements és desconeguda o dinàmica. Flexbox és ideal per crear dissenys de pàgines adaptatives, ja que facilita l'alineació, distribució i ordenació dels elements dins d'un contenidor.

1. El contenidor flex (display: flex)
El punt de partida per utilitzar Flexbox és establir display: flex en l'element contenidor. Aquest contenidor es coneix com a **contenidor flex**, i els seus elements fills es coneixen com a **elements flex**. Un cop activat el display: flex, pots utilitzar diverses propietats per ajustar com es distribueixen els elements dins del contenidor.

```css
.contingut {
  display: flex;
}
```

```html
<div class="contenidor">
  <div>Element 1</div>
  <div>Element 2</div>
  <div>Element 3</div>
</div>
```





Disseny bàsic amb Flexbox

2. Direcció del flux (flex-direction)

Per defecte, els elements flex es disposen en fila horitzontalment. Pots canviar la direcció amb la propietat flex-direction.

Els valors més comuns són:

- **row**: Els elements es col·loquen en fila (horitzontal) (valor per defecte).
- **row-reverse**: Els elements es col·loquen en fila però en ordre invers.
- **column**: Els elements es col·loquen en columna (vertical).
- **column-reverse**: Els elements es col·loquen en columna però en ordre invers.

```
.contenidor {
  display: flex;
  flex-direction: column;
}
```

Disseny bàsic amb Flexbox

3. Alineació horitzontal (justify-content)

La propietat justify-content controla l'alineació dels elements al llarg de l'eix principal (horitzontal per defecte). Pots distribuir els elements flex de diferents maneres dins del contenidor.

Els valors més comuns són:

- **flex-start**: Els elements s'alineen al començament del contenidor.
- **flex-end**: Els elements s'alineen al final del contenidor.
- **center**: Els elements es centren dins del contenidor.
- **space-between**: Es distribueixen uniformement amb espai entre ells.
- **space-around**: Els elements tenen espai al voltant de cadascun.

```
.contenidor {
  display: flex;
  justify-content: center;
}
```

Disseny bàsic amb Flexbox

4. Alineació vertical (align-items)

La propietat align-items controla l'alineació dels elements al llarg de l'eix transversal (vertical si la direcció és horitzontal). Aquesta propietat determina com s'alineen els elements flex dins del contenidor en relació amb la seva alçada.

```
.contenidor {
  display: flex;
  align-items: center;
}
```

Els valors més comuns són:

- **stretch**: Els elements s'estiren per omplir el contenidor (valor per defecte).
- **flex-start**: Els elements s'alineen a la part superior del contenidor.
- **flex-end**: Els elements s'alineen a la part inferior del contenidor.
- **center**: Els elements es centren verticalment dins del contenidor.
- **baseline**: Els elements s'alineen segons la seva línia de base de text.

Disseny bàsic amb Flexbox

5. Control de l'espai que ocupa cada element (flex-grow)

La propietat flex-grow especifica com els elements flex creixen dins del contenidor per ocupar l'espai disponible. Cada element es pot assignar un valor numèric que indica quant ha de créixer en relació amb els altres elements. Si tots els elements tenen flex-grow: 1, l'espai disponible es distribuirà de manera uniforme.

```
.element {
  flex-grow: 2; /* Aquest element creixerà el doble que els altres elements */
}
```

Disseny bàsic amb Flexbox

Resum de les propietats vistes:

1. **display: flex**: Activa el Flexbox en el contenidor.
2. **flex-direction**: Defineix si els elements es col·loquen en fila o en columna.
3. **justify-content**: Alinea els elements horitzontalment (eix principal).
4. **align-items**: Alinea els elements verticalment (eix transversal).
5. **flex-grow**: Controla com creix cada element dins del contenidor per ocupar espai.

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flexbox Bàsic</title>
  <style>
    .contenidor {
      display: flex;
      flex-direction: row; /* Col·loca els elements en fila */
      justify-content: space-around; /* Distribueix els elements amb espai al voltant */
      align-items: center; /* Centra els elements verticalment */
      height: 200px;
      background-color: lightgray;
    }

    .element {
      background-color: lightblue;
      padding: 20px;
      margin: 5px;
    }

    .element-gran {
      flex-grow: 2; /* Aquest element ocuparà el doble d'espai que els altres */
    }
  </style>
</head>
<body>
  <div class="contenidor">
    <div class="element">Element 1</div>
    <div class="element element-gran">Element 2 (Gran)</div>
    <div class="element">Element 3</div>
  </div>
</body>
</html>
```

Media Queries (Introducció)

Les **Media Queries** són una característica poderosa del CSS que permet adaptar el disseny d'una pàgina web en funció de les característiques del dispositiu que l'està visualitzant, com la mida de la pantalla, la resolució o l'orientació. Aquesta tècnica és clau per implementar **dissenys responsius** que ofereixen una experiència òptima tant en dispositius mòbils com en tauletes o ordinadors de sobretaula.

1. Què són les Media Queries?

Una **media query** és una condició que s'afegeix al CSS per aplicar regles d'estil només quan es compleixin certes característiques del dispositiu o de la finestra del navegador. L'exemple més comú és aplicar estils diferents en funció de l'amplada de la pantalla.

```
@media (condició) {
  /* Estils que només s'aplicaran si es compleix la condició */
}
```

Media Queries (Introducció)

2. Aplicació bàsica de Media Queries per a pantalles de diferent mida

Un ús típic de les media queries és adaptar el disseny a diferents amplades de pantalla. Per exemple, pots ajustar els estils per a pantalles grans i petites.

```
/* Estils per a pantalles grans (per sobre de 768px d'amplada) */
@media (min-width: 768px) {
  body {
    background-color: lightblue;
  }
}

/* Estils per a pantalles petites (per sota de 768px d'amplada) */
@media (max-width: 767px) {
  body {
    background-color: lightgreen;
  }
}
```

3. Tipus de condicions més comunes en les Media Queries

A continuació, veurem algunes de les condicions més comunes que pots utilitzar dins de les media queries:

a) Amplada de la pantalla (min-width i max-width)

Aquestes condicions s'utilitzen per aplicar estils quan la finestra del navegador té una amplada mínima (min-width) o màxima (max-width).

```
@media (min-width: 600px) {
  /* Estils per a pantalles amb una amplada de 600 pixels o més */
}

@media (max-width: 600px) {
  /* Estils per a pantalles amb una amplada de fins a 600 pixels */
}
```

Media Queries (Introducció)

b) Altura de la pantalla (min-height i max-height)
Aquestes condicions funcionen de manera similar a les d'amplada, però es basen en l'altura de la finestra.

```
@media (min-height: 500px) {
  /* Estils per a pantalles amb una alçada de 500 píxels o més */
}
```

c) Orientació de la pantalla (orientation)
Pots utilitzar la condició orientation per aplicar estils diferents si la pantalla està en orientació horitzontal (landscape) o vertical (portrait).

```
@media (orientation: landscape) {
  /* Estils per a dispositius en orientació horitzontal */
}

@media (orientation: portrait) {
  /* Estils per a dispositius en orientació vertical */
}
```

Media Queries (Introducció)

4. Ús de múltiples condicions en una Media Query

Pots combinar diverses condicions en una sola media query mitjançant les paraules clau and o only per especificar estils molt concrets.

```
@media (min-width: 600px) and (max-width: 1200px) {
  /* Estils aplicats només si la pantalla té entre 600 i 1200 píxels d'amplada */
}
```

Media Queries (Introducció)

Resum de les propietats vistes:

- **@media**: Defineix una media query.
- **min-width i max-width**: Aplica estils segons l'amplada mínima o màxima de la pantalla.
- **min-height i max-height**: Aplica estils segons l'altura mínima o màxima de la pantalla.
- **orientation**: Aplica estils segons l'orientació del dispositiu (vertical o horitzontal).
- **and**: Combina múltiples condicions en una media query.

```html
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Disseny Responsive</title>
  <style>
    body {
      font-family: Arial, sans-serif;
    }

    .contenidor {
      display: flex;
      flex-direction: column;
      padding: 20px;
    }

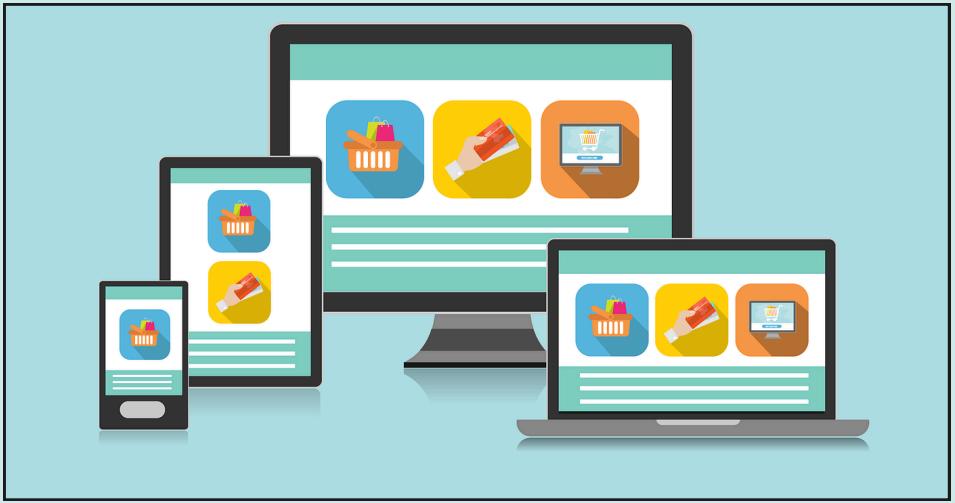
    .element {
      background-color: lightblue;
      padding: 10px;
      margin: 5px;
      text-align: center;
    }

    @media (min-width: 768px) {
      .contenidor {
        flex-direction: row;
      }

      .element {
        font-size: 24px;
      }
    }

    @media (max-width: 767px) {
      .element {
        font-size: 18px;
      }
    }
  </style>
</head>
<body>
  <div class="contenidor">
    <div class="element">Element 1</div>
    <div class="element">Element 2</div>
    <div class="element">Element 3</div>
  </div>
</body>
</html>
```

CSS AVANÇAT



Jaume Joan Pericàs Oliver
poj@paucasesnovescifp.cat

1. CSS Avançat: Introducció
2. Selectors avançats
3. Propietats avançades de fonts i tipografia
4. Layouts avançats amb Flexbox i Grid
5. Popover API
6. Variables CSS (Custom Properties)
7. Animacions i transicions
8. Disseny atractiu i coherent
9. UI/UX
10. Optimització i rendiment
11. Eines modernes de CSS

01

CSS Avançat: Introducció

A mesura que els projectes web es tornen més complexos, l'ús del CSS també ha d'evolucionar per oferir solucions més adaptatives, eficients i organitzades. El CSS avançat no només se centra en dissenyar una pàgina web atractiva, sinó també en assegurar-se que aquesta pàgina funcioni bé en diversos dispositius, sigui fàcil de mantenir i estigui optimitzada per a un rendiment òptim.

Importància del disseny adaptable i optimitzat
Un dels pilars del CSS avançat és el **disseny responsive**, que permet que les pàgines web s'adaptin a diferents mides de pantalla i dispositius. Aquesta tècnica és cada cop més important a causa de l'augment de l'ús de dispositius mòbils. Per aconseguir un disseny adaptatiu, el CSS avançat incorpora tècniques com:

1. **Media Queries avançade**
2. **Unitats relatives i flexibles**
3. **Layouts responsius amb Flexbox i Grid**

Millors pràctiques d'organització del CSS
A mesura que els projectes es fan més grans, és essencial mantenir una bona organització del codi CSS per evitar que es torni difícil de gestionar i mantenir. A continuació, algunes millors pràctiques per a l'organització del CSS en projectes avançats:

1. **Modularització del CSS**:
    a. **Dividir el codi CSS en fitxers més petits i temàtics**
    b. **Arquitectura CSS escalable (SMACSS, BEM)**
2. **Pre-processadors CSS**:
    a. **Sass** o **LESS**
3. **Optimització del CSS**:
    a. **Minificació del CSS**
    b. **Carregament asíncron del CSS**
4. **Utilització de frameworks CSS**:
    a. **Bootstrap**
    b. **Tailwind CSS**

CSS AVANÇAT

Selectors avançats en CSS

Els **selectors avançats** en CSS permeten aplicar estils amb molta més precisió i flexibilitat, permetent seleccionar elements específics en funció de la seva posició, relació amb altres elements o atributs.

1. Selectors d'atribut

Els selectors d'atribut permeten aplicar estils als elements HTML que contenen un determinat atribut o valor d'atribut. Són especialment útils per estilitzar elements de formularis o en casos en què utilitzis atributs personalitzats.

Exemples comuns:

Seleccionar elements amb un atribut específic:

```
input[type="text"] {
  border: 1px solid blue;
}
```

En aquest exemple, tots els elements <input> que tenen l'atribut type="text" tindran una vora blava.

Seleccionar elements amb un atribut que conté un valor específic:

```
a[href*="example"] {
  color: green;
}
```

Aquest exemple aplica el color verd a tots els enllaços (<a>) que contenen la paraula "example" en l'atribut href.

Seleccionar elements amb un valor d'atribut que comença amb un valor específic:

```
img[src^="https"] {
  border: 2px solid black;
}
```

Això aplica una vora negra a totes les imatges (<img>) que tenen una URL src que comença per "https".

Seleccionar tots els elements amb un atribut:

```
[personalitzat] {
  border: 1px solid red;
}
```

En aquest exemple, tots els elements que tenen l'atribut personalitzat tindran una vora vermella.

Selectors avançats en CSS

2. Selectors combinats

Els selectors combinats permeten seleccionar elements que tenen una relació específica amb altres elements dins del document. Els combinadors més comuns són:

a) Combinador de descendents ( )

Selecciona tots els elements que són descendents (fills, nets, etc.) d'un element pare.

```
div p {
  color: red;
}
```

Això aplica el color vermell a tots els paràgrafs (<p>) que són descendents de qualsevol element .

b) Combinador de fills directes (>)

Selecciona només els fills directes d'un element pare.

```
div > p {
  color: blue;
}
```

En aquest cas, només els paràgrafs que són fills immediats d'un tindran el text de color blau. Els nets o altres descendents no es veuran afectats.

c) Combinador de germans adjacents (+)

Selecciona l'element que està just després d'un altre element germà..

```
h1 + p {
  font-weight: bold;
}
```

Aquest exemple fa que el primer paràgraf (<p>) que segueix directament un títol (<h1>) es mostri en negreta.

d) Combinador de germans generals (~)

Selecciona tots els germans que segueixen un element determinat dins del mateix pare.

```
h2 ~ p {
  color: gray;
}
```

Aquest exemple fa que tots els paràgrafs (<p>) que segueixen un <h2> dins del mateix contenidor tinguin el text de color gris.

Selectors avançats en CSS

3. Selectors pseudo-classe

Les pseudo-classes són selectors que s'utilitzen per aplicar estils a un element en un estat o condició particular. Això permet canviar l'estil d'un element quan l'usuari hi interactua o quan l'element compleix certs criteris.

:hover: Aplica estils quan l'usuari passa el ratolí per sobre de l'element.

```
a:hover {
  color: red;
}
```

Aquest exemple canvia el color d'un enllaç a vermell quan l'usuari passa el cursor per sobre.

:first-of-type: Selecciona el primer element del seu tipus dins del seu pare.

```
p:first-of-type {
  font-weight: bold;
}
```

En aquest exemple, el primer paràgraf dins de cada element pare tindrà el text en negreta.

:nth-child(): Selecciona elements en funció de la seva posició entre els seus germans.

```
tr:nth-child(2) {
  color: blue;
}
```

Això aplica un color de fons gris clar a les files imparells d'una taula.

:not(): Selecciona tots els elements excepte aquells que coincideixin amb el selector especificat.

```
div:not(.important) {
  background-color: yellow;
}
```

- Això aplica un color de fons groc a tots els que no tenen la classe important.

Selectors avançats en CSS

4. Selectors pseudo-element

Els pseudo-elements permeten aplicar estils a una part específica d'un element, com el seu contingut abans o després, o una línia de text. Els pseudo-elements més comuns són **::before i ::after.**

::before: Afegeix contingut o estils abans del contingut real d'un element.

```
h1::before {
  content: "★ ";
  color: gold;
}
```

Aquest exemple afegeix una estrella daurada abans de cada <h1>.

::after: Afegeix contingut o estils després del contingut real d'un element.

```
p::after {
  content: " (Final)";
  color: gray;
}
```

Això afegeix la paraula "(Final)" en gris després de cada paràgraf.

Selectors avançats en CSS

En aquest exemple:

- Els camps de text amb type="text" tenen una vora blava.
- Els paràgrafs fills directes de div tenen el text de color verd.
- Els enllaços es tornen vermells quan l'usuari passa el cursor per sobre.
- Els títols <h1> tenen una icona al davant gràcies a ::before.

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Selectors Avançats</title>
  <style>
    /* Selectors d'atribut */
    input[type="text"] {
      border: 2px solid blue;
    }

    /* Combinador de fills directes */
    div > p {
      color: green;
    }

    /* Pseudo-classe hover */
    a:hover {
      color: red;
    }

    /* Pseudo-element before */
    h1::before {
      content: " ◆ ";
    }
  </style>
</head>
<body>
  <h1>Selectors avançats en CSS</h1>
  <div>
    <p>Aquest paràgraf és fill directe d'un `div` i és de color verd.</p>
  </div>
  <input type="text" placeholder="Escriu alguna cosa">
  <a href="#">Enllaç amb hover vermell</a>
</body>
</html>
```

CSS AVANÇAT

Fonts i tipografia avançades

El control de la tipografia en una pàgina web és crucial per garantir una bona llegibilitat i una estètica consistent. En CSS avançat, tens accés a un conjunt de propietats que et permeten controlar diversos aspectes de les fonts, com la seva càrrega dinàmica, espaiat, pes, i línies de text. També explorarem com utilitzar fonts personalitzades amb la regla **@font-face**.

1. Tipografies web amb @font-face

La regla @font-face permet utilitzar fonts personalitzades que no estan disponibles al sistema de l'usuari. Això significa que pots carregar fonts específiques des del teu servidor o des d'una font externa com Google Fonts, garantint una coherència tipogràfica entre tots els usuaris, independentment del dispositiu o navegador que utilitzin.

```
@font-face {
  font-family: "Open Sans";
  src: url("fonts/OpenSans-Regular.woff2") format("woff2"),
       url("fonts/OpenSans-Regular.woff") format("woff");
}

body {
  font-family: "Open Sans", sans-serif;
}
```

CSS AVANÇAT

Fonts i tipografia avançades

El control de la tipografia en una pàgina web és crucial per garantir una bona llegibilitat i una estètica consistent. En CSS avançat, tens accés a un conjunt de propietats que et permeten controlar diversos aspectes de les fonts, com la seva càrrega dinàmica, espaiat, pes, i línies de text. També explorarem com utilitzar fonts personalitzades amb la regla **@font-face**.

1. Tipografies web amb @font-face

La regla @font-face permet utilitzar fonts personalitzades que no estan disponibles al sistema de l'usuari. Això significa que pots carregar fonts específiques des del teu servidor o des d'una font externa com Google Fonts, garantint una coherència tipogràfica entre tots els usuaris, independentment del dispositiu o navegador que utilitzin.

```
@font-face {
  font-family: "Open Sans";
  src: url("fonts/OpenSans-Regular.woff2") format("woff2"),
       url("fonts/OpenSans-Regular.woff") format("woff");
}

body {
  font-family: "Open Sans", sans-serif;
}
```

09

Fonts i tipografia avançades

2. Control avançat de tipografia

A més de definir fonts personalitzades, CSS ofereix propietats avançades per controlar altres aspectes tipogràfics, com l'espai entre lletres, el gruix de la font o l'alçada de les línies.

a) Propietat letter-spacing

La propietat letter-spacing permet ajustar l'espai entre les lletres d'un text, fent que el text es vegi més comprimit o espaiat.

```
h1 {
  letter-spacing: 2px; /* Augmenta l'espai entre lletres */
}
```

b) Propietat line-height

La propietat line-height controla l'alçada de les línies de text. Una alçada de línia més gran augmenta l'espai entre les línies, cosa que pot millorar la llegibilitat.

```
p {
  line-height: 1.6; /* Augmenta l'alçada de les línies */
}
```

Fonts i tipografia avançades

c) Propietat font-weight
La propietat font-weight controla el gruix del text. A més dels valors **normal** i **bold**, es poden utilitzar valors numèrics entre **100** (molt prim) i **900** (molt gruixut), oferint més flexibilitat.

```
h1 {
  font-weight: 700; /* Gruix equivalent a negreta */
}
p {
  font-weight: 300; /* Gruix més prim */
}
```

d) Propietat font-style
La propietat font-style permet definir si el text es mostrarà en estil normal, cursiva o obliqua. És útil per donar èmfasi al text.

```
em {
  font-style: italic; /* Mostra el text en cursiva */
}
```

e) Propietat text-transform
Aquesta propietat permet canviar la capitalització del text, convertint-lo tot en majúscules, minúscules o capitalitzant la primera lletra de cada paraula.
uppercase: Converteix tot el text en majúscules.
lowercase: Converteix tot el text en minúscules.
capitalize: Capitalitza la primera lletra de cada paraula.

```
em {
  font-style: italic; /* Mostra el text en cursiva */
}
```

Fonts i tipografia avançades

3. Ús del Web Open Font Format (WOFF)

WOFF i **WOFF2** són formats de font comprimits dissenyats específicament per a ús web. Aquests formats proporcionen una compressió més eficient que els formats tradicionals, millorant el temps de càrrega de les pàgines sense perdre qualitat tipogràfica. És recomanable utilitzar WOFF2 quan sigui possible, ja que és el format més modern i optimitzat.

```
@font-face {
  font-family: "Roboto";
  src: url("fonts/Roboto-Regular.woff2") format("woff2"),
       url("fonts/Roboto-Regular.woff") format("woff");
}
```

L'exemple carrega la font "Roboto" utilitzant els formats WOFF i WOFF2, garantint compatibilitat amb navegadors antics i moderns.

Fonts i tipografia avançades

Exemple complet de propietats avançades de fonts i tipografia:

En aquest exemple:

- Utilitzem la font personalitzada **Lato** carregada amb WOFF2 i WOFF.
- Les propietats letter-spacing, line-height, i font-weight s'utilitzen per controlar l'espaiat entre lletres, l'alçada de les línies i el gruix de la font.
- text-transform converteix el títol en majúscules.

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Propietats avançades de fonts</title>
  <style>
    @font-face {
      font-family: "Lato";
      src: url("fonts/Lato-Regular.woff2") format("woff2"),
           url("fonts/Lato-Regular.woff") format("woff");
    }

    body {
      font-family: "Lato", sans-serif;
      line-height: 1.6;
    }

    h1 {
      font-weight: 700;
      letter-spacing: 2px;
      text-transform: uppercase;
    }

    p {
      font-weight: 300;
      font-style: italic;
    }
  </style>
</head>
<body>
  <h1>Propietats avançades de fonts</h1>
  <p>Aquest és un paràgraf amb estil cursiva i un pes lleuger de font.</p>
</body>
</html>
```

Layouts amb Flexbox i Grid

Flexbox



Grid

Casos comuns per CSS Grid:

1. Quan vols crear una pàgina amb una estructura complexa (com una pàgina amb capçalera, barra lateral, contingut principal i peu de pàgina).
2. Quan tens un disseny en format de quadrícula amb files i columnes, com un tauler de fotos o un disseny de cartes de productes.
3. Quan vols controlar exactament on va cada element en la quadrícula (pots col·locar elements en qualsevol cel·la de la quadrícula).

Layouts amb Flexbox i Grid

Quan es tracta de crear dissenys flexibles, adaptables i organitzats, Flexbox i CSS Grid són les dues eines més poderoses del CSS. Amb elles pots controlar la disposició dels elements en la pàgina, ja sigui en una fila, columna o en quadrícules complexes, adaptant els elements fàcilment a diferents mides de pantalla i requeriments.

1. Flexbox avançat: alineació, `flex-shrink`, `align-self`

Flexbox és una eina molt flexible per crear dissenys en fila o columna, on els elements s'alineen i ajusten automàticament segons l'espai disponible.

flex-shrink: Controla com es redueix la mida d'un element quan no hi ha prou espai disponible. Valors més grans permeten que l'element es redueixi més.

```
.element {
  flex-shrink: 1; /* Permet que l'element es redueixi quan no hi ha espai suficient */
}
```

align-self: Permet que un sol element en un contenidor flex tingui una alineació diferent a la resta d'elements. Els valors poden ser flex-start, flex-end, center, baseline o stretch.

```
.element {
  align-self: center; /* Centra aquest element individualment dins del contenidor */
}
```

Layouts amb Flexbox i Grid

css-tricks.com
CSS Grid Layout Guide | CSS-Tricks
Our comprehensive guide to CSS grid, focusing on all the settings both for the grid parent container and the grid child elements.

2. Grid Layout: creació de quadrícules

CSS Grid Layout és una eina més potent que Flexbox quan es tracta de crear layouts en dues dimensions (filera i columna). Grid permet col·locar elements en una quadrícula amb molta precisió, controlant la mida de les files i columnes, i permetent que els elements s'ajustin a l'espai disponible.

Propietats clau en CSS Grid:

- **grid-template-columns i grid-template-rows**: Defineixen el nombre i la mida de les columnes i files dins del contenidor grid.
- **grid-column i grid-row**: Permeten que un element ocupi diverses columnes o files dins de la quadrícula.
- **gap**: Defineix l'espai entre les files i columnes de la quadrícula.

```
.grid {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr; /* Defineix 3 columnes amb mides flexibles */
  grid-template-rows: 100px auto; /* Defineix dues files: una fixa i una automàtica */
}
```

```
.element-gran {
  grid-column: span 2; /* L'element ocuparà 2 columnes */
  grid-row: span 1; /* L'element ocuparà només 1 fila */
}
```

```
.grid {
  gap: 20px; /* Defineix un espai de 20 píxels entre les files i columnes */
}
```

Layouts amb Flexbox i Grid

Exemple bàsic amb CSS Grid:

En aquest exemple:

1. La quadrícula té 3 columnes iguals.
2. L'element 2 ocupa dues columnes gràcies a grid-column: span 2.

```
.<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Grid Bàsic</title>
  <style>
    .grid {
      display: grid;
      grid-template-columns: 1fr 1fr 1fr; /* 3 columnes iguals */
      grid-gap: 10px; /* Espai entre les columnes */
      height: 400px;
    }

    .element {
      background-color: lightblue;
      padding: 20px;
      text-align: center;
    }

    .element-gran {
      grid-column: span 2; /* Aquest element ocuparà dues columnes */
    }
  </style>
</head>
<body>
  <div class="grid">
    <div class="element">Element 1</div>
    <div class="element element-gran">Element 2 (Gran)</div>
    <div class="element">Element 3</div>
    <div class="element">Element 4</div>
  </div>
</body>
</html>
```

Layouts amb Flexbox i Grid

3. Ús combinat de Flexbox i Grid

Tot i que Flexbox i Grid es poden utilitzar per separat, sovint és útil combinar-los per obtenir el millor de tots dos. Per exemple, pots utilitzar **Grid** per definir la disposició general d'una pàgina (files i columnes) i utilitzar **Flexbox** dins d'algunes àrees per ajustar l'alineació dels elements.

En aquest exemple:

1. El **layout general** s'estructura amb **Grid**, dividint la pàgina en columnes.
2. Dins d'una de les cel·les del Grid, s'utilitza **Flexbox** per alinear els elements en una fila.

```html
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flexbox i Grid combinats</title>
  <style>
    .grid {
      display: grid;
      grid-template-columns: 1fr 2fr;
      gap: 10px;
      height: 300px;
    }

    .element {
      background-color: lightblue;
      padding: 10px;
    }

    .flex {
      display: flex;
      justify-content: space-around; /* Flexbox dins del contenidor grid */
      align-items: center;
      background-color: lightgreen;
      height: 100%;
    }
  </stsyle>
</head>
<body>
  <div class="grid">
    <div class="element">Element 1</div>
    <div class="flex">
      <div>Flex Element 1</div>
      <div>Flex Element 2</div>
    </div>
  </div>
</body>
</html>
```

Popover API

La **Popover API** és una funcionalitat recent que permet als desenvolupadors crear contingut emergent (popovers) de manera nativa, utilitzant només HTML i CSS, sense necessitat de JavaScript. Aquesta API facilita la creació de components com menús, suggeriments o notificacions que apareixen sobre el contingut existent.

Atributs HTML rellevants

popover: Atribut global que designa un element com a popover.

1. **"auto"** (per defecte): El popover es mostra i s'oculta automàticament.
2. **"manual"**: El control de la visibilitat del popover es gestiona manualment.

popovertarget: Atribut que especifica l'ID de l'element popover que es controlarà.

popovertargetaction: Defineix l'acció a realitzar sobre el popover.

1. **"show"**: Mostra el popover.
2. **"hide"**: Oculta el popover.
3. **"toggle"**: Alterna entre mostrar i ocultar el popover.

Popover API

Explicació de l'exemple:

1. El botó amb la classe trigger té l'atribut popovertarget que apunta a l'ID popoverContent, indicant que aquest botó controlarà la visibilitat del popover.
2. El div amb l'ID popoverContent té l'atribut popover, designant-lo com a contingut emergent.
3. Quan l'usuari fa clic al botó, el popover es mostra o s'oculta automàticament.
4. L'estil aplicat al pseudo-element ::backdrop crea un efecte de desenfocament al fons quan el popover està obert.

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exemple de Popover API</title>
  <style>
    /* Estils per al botó activador */
    .trigger {
      padding: 10px 20px;
      background-color: #3498db;
      color: white;
      border: none;
      cursor: pointer;
      border-radius: 4px;
    }

    /* Estils per al contingut del popover */
    #popoverContent {
      padding: 10px;
      background-color: #333;
      color: white;
      border-radius: 5px;
      width: 200px;
      text-align: center;
    }

    /* Estils per al fons quan el popover està obert */
    ::backdrop {
      backdrop-filter: blur(3px);
    }
  </style>
</head>
<body>
  <!-- Botó que activa el popover -->
  <button class="trigger" popovertarget="popoverContent">
    Mostra informació
  </button>

  <!-- Contingut del popover -->
  <div id="popoverContent" popover>
    <p>Aquest és un popover utilitzant la Popover API!</p>
  </div>
</body>
</html>
```

Variables CSS (Custom Properties)

Les **variables CSS**, també conegudes com **Custom Properties**, permeten definir valors reutilitzables dins d'un document CSS. Aquesta característica facilita molt la gestió i manteniment de grans fulls d'estil, ja que pots centralitzar valors com colors, mides i espaiats, i reutilitzar-los en diverses regles CSS. Si necessites fer un canvi en algun d'aquests valors, només hauràs de modificar la variable en un lloc, i el canvi es propagarà a totes les parts del document on s'utilitza.

1. Definir i utilitzar variables CSS

Les variables CSS es defineixen dins de qualsevol selector utilitzant el prefix --. És habitual definir-les en el selector :root per tal que estiguin disponibles globalment en tot el document.

```css
:root {
  --color-primari: #3498db;
  --marge-petit: 10px;
}

body {
  background-color: var(--color-primari);
  margin: var(--marge-petit);
}
```

Variables CSS (Custom Properties)

2. Beneficis de les variables en grans projectes

L'ús de variables és especialment útil en projectes grans o complexos, on necessites garantir la consistència en colors, mides, fonts i altres estils. Canviar aquests valors en múltiples llocs manualment pot ser tediós i propens a errors. Les variables centralitzen aquests valors en un sol lloc.

Avantatges:

1. **Mantenibilitat**: Pots modificar fàcilment valors globals com els colors o les mides canviant només una variable.
2. **Reutilització**: Les mateixes variables es poden utilitzar en qualsevol lloc del full d'estil, estalviant temps i espai.
3. **Consistència**: L'ús de variables assegura que tots els elements que depenen d'una variable tinguin el mateix estil.

Variables CSS (Custom Properties)

3. Variables CSS amb valors per defecte

Pots proporcionar un valor per defecte a les variables CSS en cas que no estiguin definides. Això és útil si vols assegurar-te que el disseny funcioni fins i tot si la variable falla o no es troba.

```
h2 {
  color: var(--color-primari, black); /* Utilitza negre si --color-primari no està definit */
}
```

Variables CSS (Custom Properties)

Exemple complet amb variables CSS:

En aquest exemple:

1. S'utilitzen variables CSS per gestionar els colors, la mida del text i els marges.
2. El botó canvia de color quan es passa el ratolí per sobre, utilitzant les mateixes variables per a la seva transició.

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Variables CSS</title>
  <style>
    :root {
      --color-primari: #3498db;
      --color-secundari: #2ecc71;
      --marge-general: 20px;
      --mida-text: 16px;
    }

    body {
      background-color: var(--color-primari);
      color: white;
      margin: var(--marge-general);
      font-size: var(--mida-text);
    }

    h1 {
      color: var(--color-secundari);
    }

    button {
      padding: 10px 20px;
      background-color: white;
      color: var(--color-primari);
      border: none;
      cursor: pointer;
    }

    button:hover {
      background-color: var(--color-secundari);
      color: white;
    }
  </style>
</head>
<body>
  <h1>Us de Variables CSS</h1>
  <p>Aquest paràgraf utilitza variables per definir la mida del text, els colors i els marges.</p>
  <button>Botó interactiu</button>
</body>
</html>
```

Animacions i transicions en CSS

Les **animacions** i **transicions** en CSS són eines potents que permeten afegir moviment i interacció visual a les pàgines web, millorant l'experiència de l'usuari. Amb aquestes tècniques, pots controlar com els elements canvien les seves propietats (com color, posició o opacitat) de manera suau i fluida.

1. Ús de transicions (transition)

Les **transicions CSS** permeten que els canvis en les propietats d'un element, com el color, l'opacitat, la mida o la posició, siguin suaus en lloc de bruscos. Quan una propietat canvia, pots definir la velocitat del canvi utilitzant transition.

Sintaxi bàsica de transition:

```
element {
 transition: propietat durada funció-temporització retard;
}
```

1. **Propietat**: La propietat CSS que vols animar (ex. background-color, transform, opacity, etc.).
2. **Durada**: Quant de temps durarà la transició (ex. 0.5s, 2s).
3. **Funció de temporització**: Controla com serà la velocitat de la transició (ex. ease, linear, ease-in-out).
4. **Retard**: Opcional, defineix quant de temps trigarà a començar la transició (ex. 1s).

Exemple de transició simple:

```
button {
  background-color: lightblue;
  transition: background-color 0.3s ease;
}

button:hover {
  background-color: darkblue;
}
```

Animacions i transicions en CSS

2. Propietats de transició (transition-property)

Si vols animar diverses propietats simultàniament, pots especificar-les individualment amb transition-property o utilitzar all per aplicar la transició a totes les propietats que canviïn.

Exemple animant múltiples propietats:

```
div {
 width: 100px;
  height: 100px;
  background-color: lightgreen;
  transition: width 1s ease, background-color 0.5s ease-in-out;
}

div:hover {
  width: 200px;
  background-color: darkgreen;
}
```


En aquest exemple:

- Quan es passa el ratolí per sobre del div, l'amplada s'animarà durant 1 segon i el color de fons durant 0.5 segons, amb funcions de temporització diferents.

Animacions i transicions en CSS

3. Animacions clau amb @keyframes

Les animacions en CSS permeten crear moviments més complexos que les transicions. Utilitzen la regla @keyframes per definir punts intermedis del moviment o canvi de propietat, i s'apliquen a través de la propietat animation.

Sintaxi de @keyframes:

```
@keyframes nom-animacio {
  0% { propietat: valor-inicial; }
  100% { propietat: valor-final; }
}
```

Sintaxi de animation:

```
element {
  animation: nom-animacio durada funció-temporització retard cicles
      direcció;
}
```

Nom de l'animació: El nom de l'animació definida amb @keyframes.
Durada: El temps que durarà l'animació.
Funció de temporització: Controla la velocitat de l'animació (linear, ease, etc.).
Retard: Opcional, quant de temps trigarà a començar l'animació.
Cicles: Quantes vegades es repetirà l'animació (infinite per repetir-la indefinidament).
Direcció: Opcional, si l'animació es juga de manera normal o inversa (normal, reverse, alternate).

Animacions i transicions en CSS

```
@keyframes girar {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

div {
  width: 100px;
  height: 100px;
  background-color: coral;
  animation: girar 2s linear infinite;
}
```

En aquest exemple:

- L'element gira contínuament gràcies a l'animació girar, que dura 2 segons i es repeteix indefinidament (infinite).

Animacions i transicions en CSS

4. Propietats d'animació (animation-duration, animation-delay)

Les propietats d'animació et permeten personalitzar com es comporta una animació. Les més importants són:

- **animation-duration**: Controla quant de temps dura l'animació.
- **animation-delay**: Defineix un retard abans que l'animació comenci.
- **animation-iteration-count**: Controla quantes vegades es repetirà l'animació (1, 3, infinite).
- **animation-direction**: Defineix si l'animació es juga cap endavant (normal), cap enrere (reverse) o alterna (alternate).

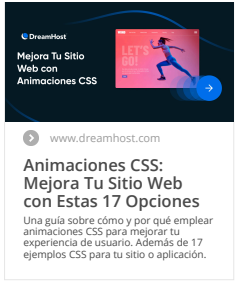
```css
/* Definició de l'element a animar */
.quadre {
  width: 100px;
  height: 100px;
  background-color: #3498db;
  position: relative;
  animation-name: moure;
  animation-duration: 2s;         /* Durada de l'animació */
  animation-delay: 1s;            /* Retard abans de començar */
  animation-iteration-count: 3;   /* Repetició de l'animació */
  animation-direction: alternate; /* Direcció alternada */
}

/* Definició de l'animació */
@keyframes moure {
  0% {
    left: 0;
  }
  100% {
    left: 200px;
  }
}
```

Animacions i transicions en CSS

En aquest exemple:

1. El botó canvia de color de manera suau quan es passa el ratolí per sobre gràcies a la transició.
2. El div es mou cap endavant i cap enrere de manera fluida gràcies a l'animació moure, que es repeteix indefinidament.



Premis disseny web.

thefwa.com

Awards - The FWA

```html
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Animacions i Transicions en CSS</title>
  <style>
    /* Transició simple */
    button {
      padding: 10px 20px;
      background-color: lightblue;
      border: none;
      cursor: pointer;
      transition: background-color 0.3s ease;
    }

    button:hover {
      background-color: darkblue;
    }

    /* Animació clau */
    @keyframes moure {
      0% { transform: translateX(0); }
      50% { transform: translateX(100px); }
      100% { transform: translateX(0); }
    }

    div {
      width: 100px;
      height: 100px;
      background-color: coral;
      animation: moure 2s ease-in-out infinite;
    }
  </style>
</head>
<body>
  <button>Canvia color</button>
  <div></div>
</body>
</html>
```

Disseny atractiu i coherent

Aquest punt se centra en les pràctiques recomanades per crear un disseny atractiu i coherent, facilitant la llegibilitat i la navegació per part dels usuaris.





1. Consistència en el disseny

1. **Tipografia**: Limita't a una o dues famílies de fonts. Assegura una jerarquia clara amb diferents mides per a títols, subtítols i paràgrafs.
2. **Paleta de colors**: Utilitza una paleta de colors definida. Evita colors excessivament vius que puguin cansar la vista.
3. **Espais**: Mantingues espais consistents entre seccions i elements per crear una estructura visual equilibrada.
4. **Estils de botons i enllaços**: Els botons i enllaços han de tenir estils unificats. Fes que els botons destacats utilitzin colors de la paleta per cridar l'atenció.

Disseny atractiu i coherent

2. Navegació clara i intuïtiva

1. **Barra de navegació fixa o visible**: La barra de navegació ha d'estar accessible en tot moment. En pantalles petites, fes servir menús desplegables (com el menú "hamburguesa").
2. **Pistes visuals**: Utilitza elements visuals, com icones o colors, per indicar la secció activa o l'element seleccionat.
3. **Rutes d'enllaç**: Assegura que els enllaços tenen noms descriptius i un estil que els distingeixi clarament del text normal.



Disseny atractiu i coherent

3. Optimització per a llegibilitat

- **Contrast de colors**: Utilitza un bon contrast entre el text i el fons per assegurar una llegibilitat òptima.
- **Mides de text**: Utilitza unitats relatives (rem o em) per al text, de manera que pugui adaptar-se a les preferències d'accessibilitat de l'usuari.
- **Alçada de línia**: Ajusta l'alçada de línia (line-height) per fer que els paràgrafs siguin més fàcils de llegir. Un valor entre 1.5 i 1.8 és recomanat.



Disseny atractiu i coherent

4. Optimització de la càrrega de la pàgina

- **Compressió d'imatges**: Utilitza formats optimitzats com WebP per reduir la mida de les imatges.
- **Fonts optimitzades**: Carrega només les mides de font necessàries i utilitza formats com WOFF2.
- **Minificació i càrrega condicional del CSS**: Redueix la mida dels arxius CSS i carrega només el CSS necessari per a cada secció o pàgina.





UI/UX

1. UI (Interfície d'usuari)
L'objectiu de la UI és fer que l'experiència visual sigui clara i atractiva per a l'usuari.

- **Principi de claredat**: Cada element de la interfície ha de tenir un propòsit clar. Evita elements innecessaris que puguin distreure l'usuari.
- **Contrast i jerarquia visual**: Utilitza el color i la mida dels elements per donar importància a les parts més destacades, com botons d'acció o seccions principals.
- **Feedback visual**: Assegura que els elements interactius com botons, formularis i enllaços ofereixin retroalimentació visual en ser clicats o seleccionats (com un canvi de color o efecte d'ombra).
- **Microinteraccions**: Afegeix petites animacions en interaccions, com botons que canvien de color o icones que es mouen, per fer l'experiència més agradable.

UI/UX

2. UX (Experiència d'usuari)

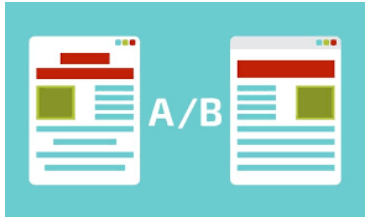
El focus del disseny UX és facilitar que l'usuari pugui aconseguir els seus objectius de manera intuïtiva.

- **Coneixement de l'usuari:** Dissenya tenint en compte les necessitats i expectatives dels usuaris. Si és un lloc de compres, per exemple, destaca els botons d'acció com "Afegir a la cistella".
- **Flux d'usuari:** Organitza els passos per aconseguir una tasca d'una manera clara. Per exemple, en un formulari de registre, és millor col·locar primer els camps essencials i després les opcions addicionals.
- **Evitar friccions:** Simplifica els processos, com la navegació o la compra. Evita passos innecessaris i proporciona una experiència fluida i sense interrupcions.
- **Adaptabilitat i responsivitat:** El disseny ha de ser adaptable a qualsevol dispositiu, ja que molts usuaris accediran des de mòbils i tauletes. Assegura't que el contingut és fàcilment accessible en pantalles petites.

UI/UX

3. Proves d'usuabilitat

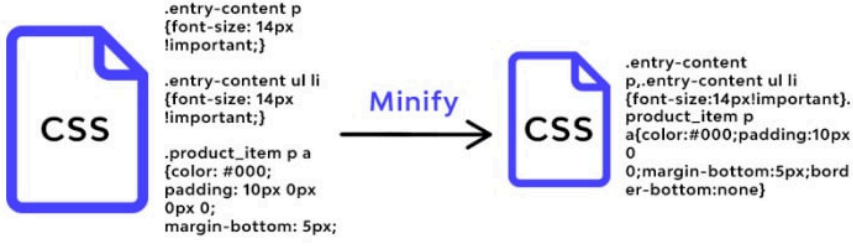
- **Feedback dels usuaris:** Recull feedback dels usuaris per entendre com interactuen amb el lloc i identificar possibles punts de millora.
- **Tests A/B:** Prova diferents versions d'elements o pàgines per veure quina ofereix una millor experiència.
- **Optimització contínua:** UX és un procés continu. A mesura que reuneixis informació sobre els usuaris i les seves preferències, aplica millores contínues per optimitzar l'experiència.





Optimització i rendiment en CSS

Optimitzar el CSS és fonamental per garantir que una pàgina web es carregui ràpidament i funcioni de manera eficient en tots els dispositius, incloent aquells amb connexions més lentes o amb menys potència de processament, com els mòbils. Una bona optimització del CSS no només millora l'experiència de l'usuari, sinó que també té un impacte positiu en el **SEO** i els rànquings de motors de cerca.



Optimització i rendiment en CSS

1. Optimització de fulls d'estil CSS

a) Minificació del CSS

La **minificació** consisteix a eliminar tots els espais en blanc, salts de línia, comentaris i altres caràcters innecessaris dels fulls d'estil CSS per reduir-ne la mida. Això ajuda a disminuir el temps de càrrega de la pàgina.
Pots utilitzar eines automàtiques com **CSSNano** o **UglifyCSS** per minificar els teus fitxers CSS.

```css
body {
  background-color: #ffffff;
  color: #333333;
}

h1 {
  font-size: 2rem;
  margin-bottom: 20px;
}
```

```
body{background-color:#ffffff;color:#333}h1{font-size:2rem;margin-bottom:20px}
```

b) Carregament condicionat del CSS

Si tens un lloc web complex amb múltiples pàgines, pots optimitzar el rendiment carregant només els estils necessaris per a cada pàgina o secció específica, en lloc de carregar tots els fulls d'estil alhora.

```
<!-- Full d'estil per a la pàgina principal -->
<link rel="stylesheet" href="main.css">

<!-- Full d'estil específic per a la pàgina de contacte -->
<link rel="stylesheet" href="contact.css" media="screen and (min-width: 768px)">
```

En aquest exemple, el full d'estil contact.css només es carregarà per a pantalles amb una amplada mínima de 768 píxels i només quan es necessiti en la pàgina de contacte.

Optimització i rendiment en CSS

2. Estratègies de minificació i compressió

A més de minificar el CSS, la compressió del fitxer també pot reduir la mida de les dades que s'envien des del servidor al navegador. Això es fa normalment a través de la compressió **Gzip** o **Brotli**, que es poden configurar al servidor.

Avantatges de la compressió:

- Redueix la mida del fitxer enviat al navegador.
- Millora els temps de càrrega, especialment per a usuaris amb connexions lentes.

Optimització i rendiment en CSS

3. Carregament asíncron del CSS

Per defecte, els navegadors bloquegen el renderitzat d'una pàgina fins que es carreguen completament els fulls d'estil CSS. Una manera d'evitar-ho és carregar el CSS de manera **asíncrona**, la qual cosa permet que la pàgina es carregui abans que es processi tot el CSS.

```
<link rel="stylesheet" href="estils.css" media="none" onload="this.media='all'">
```

En aquest exemple, el full d'estil es carrega asíncronament amb l'atribut media="none" i s'activa quan el full d'estil s'ha carregat completament, gràcies a l'atribut onload.

Optimització i rendiment en CSS

4. Ús d'eines com Autoprefixer

Els navegadors tenen diferents nivells de suport per a noves propietats CSS, com Grid Layout o **Flexbox**. L'eina **Autoprefixer** afegeix automàticament els prefixos necessaris (com -webkit-, -moz-) per garantir que les propietats CSS funcionin correctament en tots els navegadors.




```css
.flex-container {
  display: flex;
}
```

```css
.flex-container {
  display: -webkit-box;
  display: -ms-flexbox;
  display: flex;
}
```

Amb Autoprefixer, assegures que el CSS sigui compatible amb versions antigues dels navegadors sense haver d'afegir manualment tots els prefixos.

Optimització i rendiment en CSS

5. Optimització de fonts

Les fonts web poden ser una part significativa de la càrrega d'una pàgina. Algunes estratègies per optimitzar l'ús de fonts inclouen:

a) Ús de formats de fonts optimitzats (WOFF, WOFF2)
Els formats WOFF i **WOFF2** són els més eficients per a fonts web, ja que estan dissenyats per a la compressió i per ser ràpidament descarregats i processats pel navegador.

b) Subconjunt de fonts
Si només utilitzes un subconjunt limitat de caràcters d'una font (com només l'alfabet anglès), pots generar una versió reduïda de la font que només inclogui aquests caràcters, reduint així la mida del fitxer.

6. Anàlisi de rendiment en CSS

Per optimitzar completament el CSS, és important analitzar com s'està utilitzant realment el full d'estil en la pàgina. Eines com Google Lighthouse, Chrome DevTools i CSS Stats t'ajuden a:

- **Detectar CSS no utilitzat**: Elimina les regles CSS que no s'utilitzen per reduir la mida del full d'estil.
- **Identificar regles duplicades**: Elimina estils que es defineixen múltiples vegades.
- **Auditar el rendiment del CSS**: Identifica quins fulls d'estil poden estar retardant el rendiment de la pàgina.

Exemple d'ús de Chrome DevTools per trobar CSS no utilitzat:

1. Obre Chrome DevTools (F12).
2. Ves a la pestanya **Coverage** (Cobertura).
3. Carrega la pàgina i mira quines parts del CSS no s'estan utilitzant.

Aquesta tècnica t'ajuda a identificar i eliminar estils innecessaris, millorant la velocitat de càrrega.

Eines modernes de CSS

Les eines modernes de CSS són eines que faciliten i milloren el flux de treball en el desenvolupament web. Aquestes eines inclouen pre-processadors, frameworks, i altres tecnologies que permeten escriure CSS de manera més eficient, modular i escalable. A continuació, veurem algunes de les eines més populars: **pre-processadors CSS i frameworks CSS** en frameworks moderns.









Eines modernes de CSS

Sass (Syntactically Awesome Style Sheets)

Sass és un pre-processador CSS que ofereix funcionalitats avançades per millorar l'eficiència i mantenibilitat del codi CSS. Es tracta d'una extensió de CSS que afegeix característiques com variables, nestat (estructures de nivells), mixins, funcions i herència, que ajuden els desenvolupadors a escriure CSS de manera més ràpida i organitzada.

Instal·lar **Sass** és senzill i pots fer-ho utilitzant diferents eines segons les teves preferències i entorn de treball. A continuació, t'explico diverses maneres d'instal·lar Sass, pas a pas:

```
npm install -g sass
```

```
sass estils.scss estils.css
```



PARÈNTESIS

Node.js

És un entorn que permet executar codi JavaScript fora del navegador, utilitzant el motor V8 de Google Chrome, ideal per construir aplicacions de backend, eines de desenvolupament i aplicacions de temps real.



npm és l'eina que et permet instal·lar, gestionar i compartir llibreries i eines per treballar amb JavaScript i Node.js, facilitant el desenvolupament de projectes de manera eficient.

Eines modernes de CSS

style.sass

```
$color-primari: #3498db
body
  background-color: $color-primari
  font-family: Arial, sans-serif

  h1
    color: white
    font-size: 2rem
```

style.scss

```
$color-primari: #3498db;
body {
  background-color: $color-primari;
  font-family: Arial, sans-serif;

  h1 {
    color: white;
    font-size: 2rem;
  }
}
```

Eines modernes de CSS

Quin triar?

1. **Tria SCSS** si:
    - Prefereixes una sintaxi similar al CSS tradicional.
    - Estàs treballant amb un equip que utilitza codi CSS estàndard.
    - Vols mantenir la compatibilitat directa amb el codi CSS.
2. **Tria Sass** si:
    - Prefereixes una sintaxi més neta i minimalista.
    - Busques reduir el nombre de caràcters necessaris al codi.

Nota: Tant SCSS com Sass compilen al mateix CSS estàndard, per la qual cosa pots utilitzar qualsevol de les dues sintaxis segons la teva preferència personal. Avui dia, SCSS és la sintaxi predeterminada i més àmpliament utilitzada.

Eines modernes de CSS

Variables:
Permeten definir valors reutilitzables, com colors, mides o fonts, que pots utilitzar a tot el teu codi.

```
$color-primari: #3498db;
$color-secundari: #2ecc71;

body {
  background-color: $color-primari;
  color: $color-secundari;
}
```

Nesting (Nidificació):

- Permet escriure el CSS en una estructura jeràrquica que reflecteix l'HTML.

```
nav {
  ul {
    margin: 0;
    padding: 0;
    li {
      list-style: none;
      a {
        text-decoration: none;
        color: $color-primari;
      }
    }
  }
}
```

Eines modernes de CSS

Mixins:
Blocs de codi reutilitzables que es poden incloure a diferents selectors.

```scss
@mixin botó($color-fons, $color-text) {
  background-color: $color-fons;
  color: $color-text;
  padding: 10px 20px;
  border-radius: 5px;
  text-transform: uppercase;
}

.btn-primari {
  @include botó(#3498db, white);
}

.btn-secundari {
  @include botó(#2ecc71, black);
}
```

Herència (Extends):
Permet reutilitzar estils d'una classe en altres.

```scss
%botó-base {
  display: inline-block;
  padding: 10px 20px;
  border-radius: 5px;
  text-align: center;
}

.btn-primari {
  @extend %botó-base;
  background-color: $color-primari;
  color: white;
}

.btn-secundari {
  @extend %botó-base;
  background-color: $color-secundari;
  color: black;
}
```

Eines modernes de CSS

Control (if/else, bucles):
Afegir lògica al CSS.

```
$max-columns: 4;

@for $i from 1 through $max-columns {
  .column-#{$i} {
    width: 100% / $i;

    @if $i == 1 {
      background-color: lightblue;
    } @else if $i == 2 {
      background-color: lightgreen;
    } @else {
      background-color: lightgray;
    }
  }
}
```

Avantatges de Sass

Organització millorada: Amb funcionalitats com variables i nesting, pots mantenir el codi organitzat i fàcil de mantenir.
Reutilització de codi: Mixins i herència permeten evitar repeticions, reduint el temps de desenvolupament.
Escalabilitat: Ideal per a projectes grans gràcies a la seva modularitat, que permet dividir el CSS en múltiples arxius.
Compatibilitat: Compila a CSS estàndard, de manera que funciona a tots els navegadors.

Eines modernes de CSS

{less}

LESS: Un altre pre-processador CSS

LESS (Leaner Style Sheets) és un pre-processador CSS que permet utilitzar funcionalitats avançades com variables, mixins, funcions i estructures anidades per escriure CSS de manera més eficient i mantenible. Va ser un dels primers pre-processadors que van guanyar popularitat en la comunitat de desenvolupadors.

```
npm install -g less
```

```
lessc estils.less estils.css
```

Variables:

- Les variables es defineixen amb el símbol @ i poden emmagatzemar valors reutilitzables com colors, mides o fonts.

```
@color-primari: #3498db;

body {
  background-color: @color-primari;
  color: white;
}
```

Mixins:

- Un mixin és un bloc d'estil que es pot reutilitzar en diversos selectors. Es defineix com si fos una classe i es "crida" com una funció.

```
.botó-estil {
  padding: 10px 20px;
  border-radius: 5px;
  text-transform: uppercase;
}

.botó-primari {
  .botó-estil();
  background-color: #3498db;
  color: white;
}
```

Eines modernes de CSS



Nesting (Anidament):

- Permet escriure estils dins d'altres estils, seguint la jerarquia HTML

```
nav {
  ul {
    list-style: none;
    li {
      display: inline-block;
      a {
        text-decoration: none;
        color: @color-primari;
      }
    }
  }
}
```

Operacions:

- LESS permet utilitzar operacions matemàtiques directament en el codi.

```
@base-marge: 10px;

.contenidor {
  margin: @base-marge * 2; // Resultat
      : 20px
  padding: @base-marge + 5px; //
      Resultat: 15px
}
```

Eines modernes de CSS

Bootstrap 5 és un **framework de front-end** que facilita la creació de llocs web responsius i atractius sense necessitat d'escriure molt codi CSS o JavaScript. És especialment útil per desenvolupar llocs web de manera ràpida i eficient gràcies a les seves eines i components predefinits.



Característiques de Bootstrap 5

1. **Sistema de Graella (Grid System)**
   - **Estructura responsiva**: Permet crear dissenys que s'adapten a diferents mides de pantalla (mòbils, tauletes, ordinadors).
   - **Basat en Flexbox** (des de Bootstrap 4): Ofereix flexibilitat i control en l'organització d'elements.
   - **Punts de ruptura (Breakpoints)**: Classes per aplicar estils segons la mida del dispositiu (xs, sm, md, lg, xl, xxl).




```
<div class="container text-center">
 <div class="row">
    <div class="col">
      Column
    </div>
    <div class="col">
      Column
    </div>
    <div class="col">
      Column
    </div>
  </div>
</div>
```

Característiques de Bootstrap 5

2. Responsivitat integrada

- **Classes predeterminades per a dispositius mòbils**: Tots els components i utilitats de Bootstrap estan pensats per ser responsius.
- **Media Queries**: Ús de CSS avançat per ajustar estils segons la mida de pantalla.

3. Components predefinits

Bootstrap inclou una àmplia gamma de components que estalvien temps en el desenvolupament:

1. **Barres de navegació (navbar)**: Personalitzables i adaptables.
2. **Botons (btn)**: Amb estils predefinits i estats (actiu, desactivat, etc.).
3. **Targetes (card)**: Per mostrar contingut amb títols, textos, imatges i botons.
4. **Modals**: Finestres emergents fàcils de configurar.
5. **Dropdowns**: Menús desplegables per afegir funcionalitats a les pàgines.
6. **Formularis**: Amb controls avançats i estils predeterminats.

Característiques de Bootstrap 5

4. Utilitats CSS

- **Estils per a text**: Classes per ajustar mides de lletra, alineació, colors, etc.
- **Espais i marges**: Classes per gestionar padding i margin (m-3, p-2, etc.).
- **Colors**: Paleta predeterminada de colors (text-primary, bg-secondary, etc.).

5. Personalització

- **Variables CSS i preprocessadors**:
    - Compatible amb **Sass** i variables CSS per ajustar estils globals.
    - Permet personalitzar colors, fonts, i mides segons les necessitats del projecte.
- **Temes predefinits**: Bootstrap proporciona opcions fàcils per aplicar diferents estils.

6. Documentació completa

1. Bootstrap és conegut per la seva **documentació exhaustiva** amb exemples, codis, i guies per començar fàcilment.

Instal·lació de Bootstrap 5

Bootstrap 5 es pot instal·lar i integrar en projectes de diverses maneres, depenent de les necessitats del projecte i l'entorn de desenvolupament. Aquesta flexibilitat permet que sigui utilitzat tant en projectes petits com en aplicacions web més complexes.

A. CDN (Content Delivery Network)
El mètode més ràpid i senzill per incloure Bootstrap en un projecte. No cal descarregar res, només afegir enllaços al teu fitxer HTML.

B. Descarregar Bootstrap localment
Pots descarregar els arxius CSS i JS de Bootstrap i incloure'ls manualment al projecte.

C. Instal·lació amb npm
Recomanat per a projectes grans o quan treballes amb frameworks moderns com React, Angular o Vue.

Eines modernes de CSS

Tailwind CSS és un **framework CSS utilitari** que permet crear dissenys personalitzats utilitzant **classes predefinides basades en utilitats** directament a l'HTML. En lloc de proporcionar components preestilitzats (com fa Bootstrap), Tailwind ofereix eines per construir dissenys únics des de zero.
Tailwind es distingeix per la seva flexibilitat i per adoptar una filosofia **"utility-first"**, on cada classe CSS té una única funció específica, com ajustar colors, marges, espais, fonts o altres aspectes visuals.



Característiques de Tailwind

1. **Classes CSS basades en utilitats**
   - **Cada classe representa una funcionalitat específica** (com ajustar marges, colors, fonts, etc.).

2. **Altament personalitzable**
   - Tailwind utilitza un fitxer de configuració (tailwind.config.js) per ajustar:
   - Paletes de colors.
   - Tipografies.
   - Mides de pantalla i punts de ruptura.
   - Espais, marges i més.

3. **Disseny responsiu senzill**
   - Classes prefixades amb punts de ruptura per adaptar el disseny a diferents dispositius:

Característiques de Tailwind

4. Espais i mides predefinides

- Tailwind inclou una escala predefinida per a marges, espais i mides que facilita l'uniformitat del disseny.
    - Exemple: p-4 (padding de 1 rem), m-2 (margin de 0.5 rem).
- Escales predefinides:
    - Espai: p-0, p-1, p-2, ..., p-12.
    - Text: text-xs, text-sm, text-lg, text-xl, etc.
    - Colors: bg-red-500, text-blue-400.

5. Paleta de colors extensa i flexible

- Inclou una àmplia gamma de colors predeterminats amb nivells d'intensitat (del 50 al 900).

6. Control complet sobre l'estil

- Tailwind no proporciona components predefinits (com botons o targetes), sinó eines per construir-los des de zero.

Característiques de Tailwind

7. Disseny modular i optimitzat

- **PurgeCSS integrat**: Tailwind elimina automàticament el CSS no utilitzat durant la producció, fent que els fitxers CSS siguin extremadament petits.

8. Flexbox i Grid integrats

- Classes per treballar amb **Flexbox** i **CSS Grid** sense necessitat d'escriure CSS manualment.

9. Components reutilitzables

- Encara que Tailwind no ofereix components predefinits com Bootstrap, pots crear i reutilitzar classes mitjançant **@apply**.

10. Documentació completa i comunitat activa

- **Documentació oficial**: Tailwind té una documentació clara amb exemples pràctics per a cada funcionalitat. https://tailwindcss.com/
- **Comunitat activa**: Hi ha moltes eines i plantilles creades per la comunitat.